Software Architecture and Design Specification (SAD)

Perishables Management System

Project	**Perishables Management System (PMS)**
Version	1.0
Authors	Deeksha, Chris, Janhavi, and Chiranth
Date	16 October 2026
Status	Draft — architecture to be verified against implementation

Revision History

Version	Date	Author	Change summary	Approval
1.0	28 October 2026	Four-member Software Engineering project team	Initial project-specific draft	Started the Project

Table of Contents

- 1. Introduction
- 2. Architecture Goals and Constraints
- 3. Stakeholders and Concerns
- 4. Architecture and Components
- 5. Architecture Pattern and Technology Stack
- 6. Data Design
- 7. API / Service Design
- 8. Sequence Flows
- 9. Security Architecture
- 10. Error Handling, Logging and Monitoring
- 11. Traceability, Risks and Open Issues

1. Introduction

1.1 Purpose

This document describes the proposed architecture and design for PMS. It defines component responsibilities, data relationships, service boundaries, important flows, security controls, and traceability to the SRS.

1.2 Scope

The design covers product and batch management, stock movement tracking, expiry/low-stock alerts, reports, authentication, authorization, and audit history. External supplier, POS, barcode-scanner, and messaging integrations are optional future work.

2. Architecture Goals and Constraints

- Correct stock calculations and traceable changes are primary goals.
- Keep business rules testable independently of UI code.
- Use a small, understandable architecture suitable for a four-member student team and sprint delivery.
- Persist data reliably and validate all user input.
- Match final diagrams and technology descriptions to the implementation; record deviations before submission.

3. Stakeholders and Concerns

Stakeholder	Primary concern
Inventory staff	Fast stock entry, simple search, clear expiry status.
Store manager	Accurate stock position, waste reduction, actionable reports.
Administrator	User access, thresholds, auditability.
Developers	Modular code, testability, clear interfaces.

4. Architecture and Components

4.1 Chosen Pattern

Use a layered modular-monolith architecture: presentation/UI, application routes/controllers, business services, repositories/data-access, and relational database. This keeps deployment and collaboration simple while allowing service-level unit tests. A microservices architecture is not recommended for the initial prototype because it would add operational overhead without a demonstrated need.

4.2 Component Responsibilities

Component	Responsibilities	Key interactions
Presentation layer	Dashboard, forms, tables, filters, user feedback.	Calls routes/controllers; renders results.
Authentication / authorization	Sign-in/out, session or token validation, role checks.	Protects routes and service operations.
Product service	Product CRUD, category/unit validation, active status.	Uses product repository; emits audit event.
Inventory / batch service	Receive batches, validate dates/quantities, calculate stock, record adjustments and waste.	Uses batch and stock-movement repositories.
Alert service	Compute near-expiry, expired, and low-stock states from current data/settings.	Reads batch/product data; feeds dashboard and reports.
Reporting service	Inventory snapshot, expiry report, wastage report.	Queries product, batch, and movement data.
Audit service	Record important actions and actor/timestamp details.	Called by product, inventory, user-management services.
Repository / data-access layer	Encapsulate database queries and persistence.	Used by business services.
Relational database	Persist users, roles, products, batches, movements, settings, and audit events.	Accessed only through data-access layer.

4.3 Component Diagram (text specification)

Browser UI → Routes/Controllers → (Authentication & Authorization; Product Service; Inventory/Batch Service; Alert Service; Reporting Service; Audit Service) → Repositories → Relational Database. Inventory and product services call Audit Service after successful mutations. Alert and reporting services read products, batches, stock movements, and settings.

5. Technology Stack and Data Stores

Confirmed: Python; deployment: local development. UNKNOWN: web framework and database engine. Inspect the repository configuration and database setup before finalizing this section. The architecture described here is a proposed logical design and must be reconciled with actual code.

Development tooling: Git/GitHub branches and pull requests, automated checks through GitHub Actions, unit tests for business services, and a README describing environment setup. Production hosting and external integrations are outside the initial scope.

6. Data Design

Entity	Important fields	Relationships / rules
User	id, username/email, password_hash, role_id, active	Role assigned to each user; never store plaintext password.
Role	id, name, permissions	Controls protected operations.
Product	id, name, category, unit, minimum_stock, active	One product can have many batches and stock movements.
Batch	id, product_id, batch_code/reference, quantity_received, received_date, expiry_date	Belongs to one product; expiry_date must be on/after received_date.
StockMovement	id, product_id, batch_id, movement_type, quantity, reason, timestamp, user_id	Receipts increase stock; reductions/waste decrease stock; corrections require authorization.
SystemSetting	key, value, updated_by, updated_at	Stores near-expiry window and defaults.
AuditEvent	id, user_id, action, entity_type, entity_id, timestamp, summary	No passwords or secrets in event payloads.

6.1 Data Integrity Rules

- Use database constraints where supported and repeat important validation in the service layer.
- Store quantities using a suitable numeric type; do not use binary floating point for quantities where precision matters.
- Calculate available quantity consistently from stock movements or maintain a cached quantity with transactional updates and reconciliation tests.
- A stock reduction must not exceed available stock except through an explicitly authorized correction process.
- Record dates in a consistent format and apply the configured store-local date when evaluating expiry.

7. API / Service Design

The exact URL routing may follow the repository's framework. The following is a proposed service/API contract; adapt it to the implementation and document any differences.

Operation	Method / route (example)	Input	Successful result / errors
List/search products	GET /api/products	query, category, status	Product list; 400 for invalid filter.
Create product	POST /api/products	name, category, unit, minimum_stock	Created product; 400 validation error; 401/403 access error.
Receive batch	POST /api/batches	product_id, quantity, received_date, expiry_date, reference	Batch created and stock movement recorded; 400 invalid data.
Record stock reduction	POST /api/stock-movements	batch_id, quantity, movement_type, reason	Movement recorded; 400 invalid quantity; 409 insufficient stock.
Get alerts	GET /api/alerts	as_of_date (optional)	Near-expiry, expired, and low-stock lists.
Get reports	GET /api/reports/{type}	date range and filters	Report data; 400 invalid range; 403 if role is unauthorized.

8. Sequence Flows

8.1 Receive Stock Batch

- Inventory staff opens the receive-stock form and submits product, quantity, received date, and expiry date.
- UI sends the request to the route/controller.
- Controller checks authentication and role, then calls Inventory/Batch Service.
- Service validates product existence, positive quantity, and date range.
- Service writes the batch and receipt stock movement in one database transaction.
- Audit Service records the successful receipt; controller returns the created batch and updated stock summary.
- If validation or persistence fails, the transaction is rolled back and a safe error is returned.

8.2 Record Waste / Spoilage

- Staff selects a batch and enters waste quantity and reason.
- Authorization and server-side validation run.
- Inventory Service checks available quantity and writes a negative stock movement atomically.
- Audit event is recorded after the stock update succeeds.
- Dashboard and wastage report reflect the updated stock and waste record.

9. Security Architecture

Threat / concern	Control
Unauthorized access	Authentication plus server-side role checks for every protected operation.
Tampering with stock	Server-side validation, transactional updates, constraints, and audit trail.
Credential disclosure	Salted password hashing, HTTPS in deployed environments, secret-free logs.
Injection / malformed input	Parameterized database access and strict input validation.
Privilege escalation	Deny-by-default permissions and tests for direct requests by unauthorized roles.
Accidental data loss	Database backup/export procedure and documented recovery steps appropriate to the prototype.

Threat-modeling prompts: spoofing (fake user/session), tampering (quantity/date manipulation), repudiation (denial of stock adjustment), information disclosure (credential or inventory exposure), denial of service (invalid or excessive requests), and elevation of privilege (inventory role performing admin operations).

10. Error Handling, Logging and Monitoring

- Return consistent validation and permission errors; do not show stack traces to end users.
- Log unexpected failures with a correlation/reference ID where feasible; never log passwords or secrets.

- Record business audit events separately from diagnostic logs.
- Monitor application startup failures, failed operations, test results, and stock reconciliation errors.
- Provide an understandable empty state when no batches or alerts match the selected filter.

11. Traceability, Risks and Open Issues

SRS requirement	Design component
PMS-F-001 / PMS-SR-001	Authentication / authorization
PMS-F-003 to PMS-F-004	Product Service and Product Repository
PMS-F-005 to PMS-F-009	Inventory/Batch Service, StockMovement repository, database transaction
PMS-F-010 to PMS-F-012	Alert Service
PMS-F-015 to PMS-F-017	Reporting Service
PMS-F-018 / PMS-SR-005	Audit Service
PMS-NF-001	Indexes, query design, performance test suite
PMS-NF-005	Layered modular architecture and service unit tests

11.1 Risks and Mitigations

Risk	Mitigation
Incorrect stock totals	Centralize stock rules in service layer; add reconciliation and unit tests.
Expiry edge cases/time-zone errors	Define store-local date rules and test today, tomorrow, and past dates.
Merge conflicts across four contributors	Assign clear module ownership, use feature branches and pull requests, integrate frequently.
Scope expansion	Prioritize core product/batch/stock/alert flows; label optional integrations as future work.
Requirements differ from delivered product	Maintain RTM and document deviations for final demonstration.

11.2 Open Issues

- Confirm the final web framework and database from the repository.
- Confirm whether multi-store support is required; current baseline assumes one store.
- Confirm whether expiry alerts are dashboard-only or require external notifications.
- Populate the final component/class/sequence diagrams using the actual module names before submission.

Team module ownership:

Member 1 — Deeksha

Frontend / UI

Responsibilities

- Build the dashboard and inventory interface.
- Implement product listing, forms, search and filters.
- Apply the agreed UI theme and styling.
- Connect the UI to backend services or APIs.
- Handle loading states, validation messages and errors.

Deliverables: Working dashboard, product screens, inventory screens and UI-related commits.

Member 2 — Chris

Backend / Inventory Services

Responsibilities

- Implement product CRUD and inventory business logic.
- Develop batch and stock-management operations.
- Validate quantities and expiry dates.
- Implement stock adjustments and wastage calculations.
- Write unit tests for backend services.

Deliverables: Inventory service modules, validated stock operations, unit tests and backend-related commits.

Member 3 — Janhavi

Testing / Quality Assurance

Responsibilities

- Prepare test cases from the SRS requirements.
- Test product CRUD, batch management and stock calculations.
- Verify near-expiry, expired-stock and low-stock alerts.
- Test invalid inputs and insufficient-stock scenarios.
- Run regression tests and report defects.
- Maintain test evidence and update the requirements traceability matrix.

Deliverables: Test cases, test results, defect reports and testing-related commits.

Member 4 — Chiranth

CI/CD / Integration / Documentation

Responsibilities

- Configure GitHub Actions to run automated tests.
- Verify that CI checks run on pushes and pull requests.
- Integrate and verify frontend and backend changes with the team.
- Maintain the SRS, SAD and Test Plan documentation.
- Prepare architecture diagrams and document design decisions.
- Support final regression checks and demo preparation.

Deliverables: CI workflow, integration fixes, architecture documentation and CI/documentation-related commits.